Animating in Blender Guide

Callum Mosley, Vince Eke, Prof Claude

November 10, 2026

A single Python pipeline turns a directory of SWIFT giant-impact snapshots (`.hdf5`, one per frame) into a rendered animation. Stage 1 (the science environment, on COSMA compute nodes) does the physics and geometry; stage 2 (Blender's own Python) sets the look, assembles the scene and makes a template; stage 3 (also Blender, on a GPU machine) renders every frame from that template. One parameter, `RENDER_MODE`, chooses how the simulation is drawn, and the same three scripts serve every choice. This document gives an overview, then covers each stage, a checklist for a full run, and finally the Blender controls you are most likely to want if you adjust a scene by hand in the GUI.

1 What you need

1. **Blender 5.x** – download from https://www.blender.org/download/ (the GUI notes below were written against 5.1; the mad06 build is 5.3). Blender ships its own Python, so stages 2 and 3 need no package installation; OpenVDB, needed for the mist modes, is included in recent builds.

2. **Python 3.10 or later** for stage 1 with six packages. On COSMA, `module load python/3.12.4` and install with the same `python3` you will run, so that the packages land in the right place:

```
module load python/3.12.4
python3 -m pip install --user swiftsimio scikit-image scipy h5py \
                              numpy pysplashsurf
```

Elsewhere, a conda environment or venv works equally well (`python3.12 -m venv giantimpact`, `source giantimpact/bin/activate`, then the same `pip install`). `pysplashsurf`, which makes the surface meshes, needs Python 3.10 or later; with an older Python, pip silently installs an out-of-date version.

2 Overview

2.1 Body, rest and render modes

Every particle is either **body** – at least `SURFACE_RHO` ($1000\,\mathrm{kg\,m^{-3}}$) dense, inside the box, and with at least `MIN_NEIGHBOURS` body neighbours (which drops lone dense specks) – or **rest**, which is everything else. Each particle is therefore shown exactly once. The body is drawn as particles or as a surface mesh, and the rest as particles or as a glowing volume (mist); `RENDER_MODE` names the two choices, body first:

RENDER_MODE	Body	Rest
PARTICLES	particles	particles, fading out at low density
MESH_PARTICLES	surface mesh (splashsurf)	particles, fading out at low density
MESH_MIST	surface mesh (splashsurf)	mist
PARTICLES_MIST	particles	mist

Making the mesh (stage 1) and the mist (stages 1 and 2) are the expensive steps; particles are cheap.

2.2 The scripts

- `combined_01_make.py`, run in chunks on COSMA compute nodes by `stage1_slurm.sh` – reads the `.hdf5` snapshots and writes the body and rest files for the chosen mode. A few test frames can be run directly on a login node.
- `combined_02_build.py` – builds the Blender scene. Either renders a handful of test frames to iterate on the look, or (`MAKE_TEMPLATE = True`) writes the final `.blend` template and, for mist modes, the `.vdb` files.
- `combined_03_render_chunk.py` with `render_chunks.sh` – renders a range of frames from the template; the shell wrapper splits the run into chunks and runs several at once on the GPU.

2.3 Where the files go

All outputs live in the run directory `BASE_DIR/SIM_NAME/`. Each file depends only on its own style, so modes share files: `MESH_PARTICLES` reuses the meshes of a `MESH_MIST` run, `PARTICLES_MIST` reuses its volume and `.vdb` files, and `PARTICLES` reuses the body particles of `PARTICLES_MIST` and each stage remakes only the files whose settings have changed (section 7).

	Particles	Mesh / mist
Body	`particles/body_NNNN.ply`	`meshes/body_NNNN.ply`
Rest	`particles/rest_NNNN.ply`	`volume/volume_NNNN.npz` (stage 1) → `vdb/mist_NNNN.vdb` (stage 2)

Stage 2 test renders go to `renders/`, and its template is `SIM_NAME_mode_camera.blend` (e.g. `..._mesh_mist_persp.blend`, or `..._mesh_mist_vr360_3d.blend` for stereo), so templates for different modes and cameras sit side by side. A test run saves `..._test.blend` instead, so it never overwrites a template. Stage 3 renders go to `renders_mode/`, named `frame_NNNN.png`, or `fish_NNNN.png` for the fisheye camera and `vr360_NNNN.png` for the 360-degree camera, with `_3d` added for stereo (e.g. `vr360_3d_NNNN.png`).

2.4 Workflow

1. Stage 1 on a few test frames (login node, or a short batch job).
2. Stage 2 test renders (`MAKE_TEMPLATE = False`); adjust stage-2 parameters, including the colours, until happy.
3. Stage 1 on all frames with `stage1_slurm.sh`.
4. Stage 2 once more with `MAKE_TEMPLATE = True` to write the template (and all `.vdb` files).

5. Render one short chunk (e.g. frames 0–2) with stage 3 and check it, then the whole run with `render_chunks.sh`; then make the movie (stage 4).

Frame numbering. Frame 0 is the initial-conditions file and frame N the Nth snapshot. Blender numbers its frames from 1, so snapshot N is Blender frame $N+1$ throughout; the camera keyframes in stage 2 use Blender frames.

Choosing frames. Stages 1 and 2 take frames on the command line in any mix of `800`, `300-319` (inclusive) and `0-2160/100` (every 100th); stage 1 also accepts `all`, or `START END` as used by the batch script. In stage 2 the frames come after Blender's `--`.

3 Stage 1 – processing the .hdf5 snapshots

Each snapshot is read and centred on its centre of mass. **One Blender unit is the simulation's own length unit**, read from the snapshots (the Earth radius for this run), so lengths in Blender read as they do in the simulation, with each frame's centre of mass at the origin. Stage 1 prints the unit, and the size of the simulation domain in the same units. Only material inside a cube of side `BOX_SIZE` is drawn. Choose it to hold everything you want to see over the whole run: each frame reports the percentage of its dense material that falls outside, as a guide. Particle masses and material IDs come from the IC file, matched by particle ID. All quantities are in SI units.

Body mesh. The surface is made by splashsurf (`pysplashsurf`) directly from the body particles: a sparse marching-cubes grid with weighted Laplacian smoothing, so its detail follows the particle resolution rather than a fixed grid, and thin tidal tails come out as clean surfaces. splashsurf's iso level is a "fluid fraction" ($\sim$1 inside any group of particles, however sparse), so it cannot tell dense material from debris itself; that is why the body is chosen by density first. Its particle radius is that of a sphere holding one particle at `SURFACE_RHO` ($\sim$77 km here), which sets the finest detail.

Rest particles fade in, through their alpha, between the two densities in `REST_FADE_RHO`; thinner material is left out. This keeps the debris localised rather than spread into a wide halo, and the gradual fade stops particles popping on and off between frames.

Mist. The volume files hold density and heat (temperature) grids of the rest particles only, made with swiftsimio's `render_gas` on a `RESOLUTION`^{3} grid; the body particles are given zero mass there.

Physics, not colour. The files carry what the material is, not how it should look: every particle and mesh vertex has its temperature (`T_K`) and iron fraction (`iron`), and the volume files hold density (kg m^{-3}) and temperature grids. All colour is decided in stage 2 (section 4), so the look can be changed without rerunning this stage. Each particle also carries a `radius_factor`, from 0.5 at `RHO_DEBRIS` to 1.0 at `RHO_BODY`, which sets its size and, in stage 2, how strongly it glows, and rest particles an `opacity`, their density fade. Stage 1 prints the 5/50/95% surface temperatures of each mesh, which are a good guide to stage 2's `GLOW_K`.

3.1 Important parameters

Parameter	Default	Effect
`SIM_NAME`, `BASE_DIR`	–	Used to find ICs, snapshots and set outputs – assumes our typical directory structure.
`RENDER_MODE`	`MESH_MIST`	See the table above. **Must equal stage 2.**
`TEST_FRAMES`	7 frames	Frames processed when none are given on the command line.

Parameter	Default	Effect
FORMAT_VERSION	1	Part of every file's fingerprint (section 7). Only needed if you edit the code itself: add one and every file is remade.
BOX_SIZE	20	Side of the box, in Blender units. A larger box holds more of the outer debris, but spreads the mist grid's RESOLUTION cells more thinly. Changing it does not change the framing, which the camera sets.
BLENDER_UNIT_M	None	Metres per Blender unit; None uses the snapshots' length unit.
SURFACE_RHO	1000	SI density dividing body from rest, in every mode.
MIN_NEIGHBOURS	4	Body particles need this many body neighbours within 2 particle radii (0 = off).
RHO_DEBRIS, RHO_BODY	1000, 2000	SI densities over which radius_factor ramps from 0.5 to 1.
REST_FADE_RHO	(5, 20)	SI densities over which rest particles fade in; lower shows more diffuse debris.
SPLASH_CUBE_SIZE	0.5	Mesh cell size in particle radii; smaller is finer, with $\sim 1/\text{size}^2$ more triangles.
SPLASH_SMOOTH_ITERS	10	Mesh smoothing passes; fewer keeps more detail, more removes per-particle bumps.
SPLASH_ISO, SPLASH_SMOOTHING_LEN, SPLASH_NORMALS_ITERS	0.6, 2.0, 10	splashsurf's iso level, kernel size and normal smoothing; rarely need changing.
RESOLUTION	256	Mist grid cells per side; a multiple of stage 2's VDB_RES. Stored in half precision (float16), which is ample for rendering; temperatures are capped at MIST_MAX_T_K (60 000 K). At 384 the volume files took 258 GB for this run, more than the snapshots.
IRON_ID	402	MaterialID of iron; others are rock.
TARGET_MAX_ID	None	Particle IDs up to this are the target, higher ones the impactor, for colouring by body (stage 2). None finds it by splitting the two bodies in the IC file, and stops if the target's IDs are not the lowest block.

3.2 Anticipated operation

For a few test frames on a cosma5 login node, limit splashsurf's threads (it otherwise uses every core):

```
env RAYON_NUM_THREADS=4 python3 combined_01_make.py 300-319
```

(env is needed in tcsh.) For a full run, submit the job array from the directory holding the scripts:

```
sbatch stage1_slurm.sh                          (frames 0–2160 in 20 chunks)
sbatch --array=0 stage1_slurm.sh 300 319        (one task, e.g. to measure memory)
```

The frame range is split evenly over the array tasks, so `--array` alone sets the number of chunks, and `--cpus-per-task`, `--mem` and `-t` on the command line override the script. Each task writes `stage1_JOB_TASK.out` and `.err`; failed frames are reported at the end of the `.out` file, and a rerun of the chunk does only what is missing or out of date.

Memory. sacct's MaxRSS includes the node's file cache, which fills up to whatever `--mem` is requested, so it is no guide. The true peak is the "Maximum resident set size" line written to the `.err` file by `/usr/bin/time -v`. COSMA5 charges about 4.8 GB of memory per core, rounding up to whole cores, so with 4 cores up to 19 GB costs nothing extra. Small, short jobs (a few cores,

under 3 hours) are backfilled quickly even when the queue is busy.

3.3 Typical resources used

For 2160 3×10^6-particle snapshots, `BOX_SIZE` = 35, `RESOLUTION` = 256, on cosma5 (remaking only the volume files took ∼22 min in 20 chunks):

Run	Request	Time and true peak memory
`MESH_MIST`, all new	20 chunks × 4 cores, 19 GB	up to 1 h 42 min per chunk (∼25 s splash + ∼15 s mist per frame); ∼10 GB
Particles only (`PARTICLES`)	10 chunks × 1 core, 4 GB	∼15 min; ∼0.8 GB
Reusing meshes or mist	10–20 chunks × 1–4 cores, 4 GB	15–50 min (more cores speed up the neighbour search); ∼0.9 GB

Disk: meshes 10–45 MB per frame (57 GB), volume files ∼9 MB per frame in half precision (20 GB; 258 GB at `RESOLUTION` = 384 in full precision with a smaller box), particle files ∼35 MB per frame per component (∼77 GB for all particles).

4 Stage 2 – setting the scene using Blender

Stage 2 reads the stage-1 files for the chosen mode and turns them into a configured Blender scene: it imports the geometry, builds the materials (`BodyLook` for the mesh, `ParticleLook` for all particles, `MistLook` for the volume), sets up the Geometry Nodes that draw each particle as a sphere of radius `PARTICLE_RADIUS` × `radius_factor`, the lighting (`KeySun` and `FillSun`), and the camera and its keyframes. Particles are lit by the suns as well as glowing, and partly transparent, so cool debris still shows and debris in front of the body only tints it.

With `MAKE_TEMPLATE = False` it renders the test frames to `renders/` (with EEVEE, except for the fisheye camera) and saves a `.blend` that still contains those frames – the one to open in the GUI. With `MAKE_TEMPLATE = True` it builds the `.vdb` files (mist modes), renders `TEMPLATE_FRAME` as a check, removes all per-frame objects and saves the template. The template keeps the materials, camera, lights and a hidden `PointsTemplate` object carrying the Geometry Nodes, and records the mode, the run directory and the component files, so stage 3 needs no settings of its own. The `.vdb` files hold the mist's density (`rho`) and mass-weighted temperature (`T_K`).

Colour. One node group, `HeatLook`, colours the body, the particles and the mist alike, from temperature and iron fraction, so material looks the same whichever side of the body/rest split it falls. Every surface (and the mist, as dust) has its material's own colour – rock or iron – lit by the suns. Hot material also glows, in the `PALETTE` colour for its temperature, with a strength rising smoothly (in $\log T$) from 0 to 1 across `GLOW_K`. So the cool crust stays dark and only hot material, such as cracks and the impact site, shines. Colours are given as display (sRGB) values, as in a colour picker. Since the colour is set here, it can be tuned with test renders alone.

4.1 Important parameters

Parameter	Default	Effect
		General
`SIM_NAME`, `BASE_DIR`	–	**Must equal stage 1.**
`RENDER_MODE`	`MESH_MIST`	**Must equal stage 1**; also sets the template filename.

Parameter	Default	Effect
BODY_ON, REST_ON	True	Include the body and the rest in the test renders; REST_ON = False in a mist mode also skips building .vdb files.
MAKE_TEMPLATE	False	False: render the test frames. True: make the template (and .vdb files), checking with TEMPLATE_FRAME. Stage 3 only accepts a template.
TEST_FRAMES, TEMPLATE_FRAME	–	Test frames (or give them after --), and the check frame in template mode.
CAMERA_MODE	PERSP	Perspective (PERSP, 1280×720), orthographic (ORTHO), planetarium (FISHEYE, 4096×4096, Cycles) or 360-degree for a headset (VR360, 4096×2048, Cycles). The path is the PATH list of (Blender frame, camera position relative to the centre, in Blender units); Blender eases smoothly between them.
CAMERA_VIEW	(1.0, 0, None)	Adjusts PATH for this template: (distance scale, turn about the spin (z) axis in degrees, elevation above the collision plane in degrees or None for as in PATH). One path then serves every projection, e.g. (0.22, 0, 70) for the dome, looking down on the collision, and (0.22, 180, 45) for a headset. camera_clearance.py reports, for candidate views, how close dense material comes to the camera; on a dome or in a headset, material passing close is uncomfortable. Not used by ORTHO.
DOME_TILT_DEG, DOME_AZ_DEG	40, 10	Tilt and azimuth of the fisheye view.
STEREO	False	Render left and right views into one image: side by side for PERSP and FISHEYE, top-bottom for VR360 (the format headset video players expect). Not for ORTHO. Doubles the render time.
STEREO_DEPTH	1/30	Eye separation as a fraction of the camera distance (kept as the camera moves); larger gives stronger 3D. Material at the box centre appears at screen depth.
TIME_LABEL	None	Optional label burned into the top-left corner of every frame, e.g. " t = {hours:.1f} h"; {seconds}, {hours} and {days} are the snapshot time, taken as the number in its file name times SNAPSHOT_NAME_S. TIME_LABEL_SIZE sets the font size in pixels. Best with PERSP or ORTHO: on the dome or headset that corner is out of view.
	Colour (body, particles and mist)	
ROCK_RGB, IRON_RGB	–	The materials' own colours, lit by the suns; darker values give a darker crust.

Parameter	Default	Effect
GLOW_K	(2800, 16000)	Glow strength rises smoothly (in $\log T$) from 0 to 1 between these temperatures. Raise them to keep the glow to the hottest material; the stage-1 surface temperature percentiles are a guide. May instead be {snapshot frame: (low, high), ...}, e.g. {0: (2800, 16000), 2160: (5000, 25000)}: the range then changes smoothly between those frames, and the template carries the animation into stage 3. PALETTE_SHIFT, BODY_EMIT_GAIN, EMIT_GAIN and MIST_EMIT_GAIN can change with time in the same way.
PALETTE_SHIFT	1.0	The glow colour is taken from PALETTE at temperature × this factor, so values above 1 make glowing material whiter without changing which material glows. May change with time, e.g. {0: 3, 300: 3, 600: 1} for white-hot cracks during the impact only.
PALETTE	2500–40000 K	Temperature (K) → glow colour stops, interpolated in $\log T$; the top stop (40 000 K) is white-hot, reached by the hottest mist or with PALETTE_SHIFT > 1.
Particles (body and rest)		
PARTICLE_RADIUS	0.005	Base sphere radius (Blender units), scaled by radius_factor (0.5–1). 0.004–0.005 keeps the mantle opaque over the core.
EMIT_GAIN	1.0	Glow strength of the hottest dense particles.
DEBRIS_EMIT_FRAC	0.3	Fraction of that glow for the least dense particles (radius_factor 0.5), rising to 1 at RHO_BODY.
PARTICLE_ALPHA	0.7	Opacity, multiplied by the rest particles' density fade. Lower is gassier, but lets the core show through and dims the glow.
PARTICLE_COLOURS	HEAT	HEAT: material colour, lit, glowing with temperature. BODY: a fixed colour for each of target and impactor rock and iron (BODY_COLOURS), with a little self-glow (BODY_GLOW), to show where the impactor's material ends up.
CUTAWAY	None	Remove the particles on the near side of a plane through the target's centre (the centre of mass of its dense material, recorded by stage 1 for each frame), showing the layers inside. "CAMERA": the plane faces the camera and turns with it; or the plane's normal in simulation axes, e.g. (0, 0, 1) removes $z > 0$. Particles only: meshes and mist are not cut, so use PARTICLES mode.
Mesh body		
BODY_EMIT_GAIN	1.0	Glow strength of the hottest surface.
Mist		
VDB_RES	128	Mist grid cells per side in the .vdb files.
MIST_DENSITY, MIST_EMIT_GAIN	0.5, 0.05	Opacity of the densest mist, and glow of the densest, hottest mist. Cycles (stage 3) shows the mist differently from EEVEE (test renders), so check it in a stage-3 render.

Parameter	Default	Effect
MIST_FADE_RHO	(1, 1000)	Mist opacity and glow rise smoothly (in log density) from zero to full between these densities ($\mathrm{kg\,m^{-3}}$), as REST_FADE_RHO does for particles: the mist is strongest next to the body and fades away in thin, distant material. Raise the lower value to pull the mist in towards the body. Set in the material, so changing it needs no new volume or .vdb files.
VDB_SMOOTH	2	Smoothing passes on the mist grid when writing the .vdb files.

4.2 Anticipated operation

On mad06, or another machine with GPUs,
`blender -b -P combined_02_build.py -- 300-319`
renders those test frames; repeat until happy with the look. All of this tinkering can also be done in the Blender GUI, but it is slow and has a great many options; there are some hints in section 9. When content, set `MAKE_TEMPLATE = True` to make the template for the whole simulation. To test the whole pipeline quickly on a mist mode, run stage 1 on frames `800-805` only, set `TEMPLATE_FRAME = 800`, make the template (`.vdb` files are built only where volume files exist) and render just those frames in stage 3. If two modes' stage-2 test runs are made at once, their test renders overwrite each other in `renders/`; the templates are safe.

4.3 Typical resources used

Test frames take seconds each. Writing the `.vdb` files for a mist mode takes ∼30 min for 2160 frames (from half-precision volume files; 1 h 50 min before) at `VDB_RES` = 128 and fills ∼15 GB; this is only paid again for files whose volume, `VDB_RES` or `VDB_SMOOTH` has changed, and `PARTICLES_MIST` and `MESH_MIST` share the same files. Even with every `.vdb` file up to date, a template run takes several minutes, up to ∼15 min when the shared filesystem is busy, because it checks every file; it reports progress every 250 frames. Fisheye and 360-degree check frames render in Cycles on the GPU with 16 samples.

5 Stage 3 – rendering using Blender

`combined_03_render_chunk.py` opens the template and, for each frame in the range given on the command line, imports that frame's files, attaches the template's materials and Geometry Nodes, renders a `.png` with Cycles on the GPU, and deletes the frame's objects again. It reads the mode and component files from the template, and stops with an error if the `.blend` was saved in test mode or by an older stage 2. `render_chunks.sh` splits a whole run into chunks and runs several Blender processes at once.

5.1 Important parameters

Parameter	Default	Effect
Shell wrapper: `render_chunks.sh`		

Parameter	Default	Effect
SIM_NAME, MODE, CAMERA	–	Pick the template SIM_NAME_MODE_CAMERA.blend. MODE is the lower-case RENDER_MODE, e.g. mesh_mist, and CAMERA the lower-case CAMERA_MODE, e.g. persp, with _3d added for stereo; **check these match the template you just made.**
START_FRAME, END_FRAME, CHUNK	0, 2160, 100	Range of frames, and frames per Blender process.
MAX_PARALLEL	8	Concurrent Blender processes, limited by GPU memory (∼4 GB each).
NTHREAD	10	CPU threads per Blender process; MAX_PARALLEL×NTHREAD ≤ 128 on mad06.
BLENDER_BIN, OUT_DIR, RENDER_SCRIPT	–	Blender executable, log directory, and path to combined_03_render_chunk.py.
Blender renderer: combined_03_render_chunk.py		
BODY_ON, REST_ON	True	Switch components off at render time without rebuilding the template.
SAMPLES	16	Cycles samples (denoising is on); the main speed-versus-noise setting.
SMOOTH_BODY	True	Smooth-shade the mesh body.

5.2 Anticipated operation

First render a few frames by hand and check them, e.g.
`blender --factory-startup -t 8 -b TEMPLATE.blend -P combined_03_render_chunk.py -- 0 2`
More than one frame is needed to check that each frame's objects are cleared properly. Then run the whole range on mad06 with
`./render_chunks.sh` (or `./render_chunks.sh 300 599` for a sub-range)
or through the queue with `sbatch stage3_slurm.sh`, a job array that gives each task a whole GPU node in `hardwarelab-gpu` and runs `render_chunks.sh` on its share of the frames there (`--array=0-3` uses up to four nodes; `sbatch -w mad06 --array=0 stage3_slurm.sh` keeps to mad06). `MAX_PARALLEL` (default 8) sets the renders per GPU, and their threads share out the node's cores; use ∼5 for `PARTICLES` modes. Each task asks for one GPU and 64 cores rather than a whole node, so others can use the rest. On a GPU type new to the Blender build (mad06's build was compiled for its A100), Cycles first compiles its GPU code, which takes 4–6 min per process; it is then cached in `~/.cache/cycles` and reused on every node, so render one frame alone on a new GPU type first. The H100 (gn004) and RTX Pro 6000 (gn005) work this way; ga008's AMD GPUs are not supported by the build and are excluded.
and watch the GPU with `watch -n 2 nvidia-smi`. Each chunk writes a log to `OUT_DIR`. Lines about `libur_loader.so.0: no version information available` and `HIPEW initialization failed` come from the oneAPI and AMD libraries and can be ignored. Afterwards, check that there is one `.png` per frame and that no log reports running out of memory:

```
ls renders_MODE/frame_*.png | wc -l
grep -il "out of memory" OUT_DIR/render_*.log
```

A chunk that failed can be re-rendered on its own by giving its frame range after `--` as above.

5.3 Typical resources used

Each Blender process takes ∼4 GB of GPU memory; mad06 has 40 GB in total, shared with other users, and 128 cores. Each frame spends part of its time on the CPU (importing files), so the GPU is idle about half the time and more processes help, as far as memory allows; raising `SAMPLES` also costs less than proportionally. For 2160 frames of 3×10^6 particles:

Mode	`MAX_PARALLEL` × `NTHREAD`	**Time**
`MESH_MIST` (new colour model, half-precision mist)	10 × 12	42 min (3 × 16 took 105 min with the old files)
`PARTICLES`	5 × 10	56 min
`PARTICLES_MIST`	8 × 10	37 min
`MESH_MIST`, `FISHEYE` 4096^2	10 × 12, `CHUNK` 50	4 h 20 min; ∼10 frames failed for lack of GPU memory (3.6–4 GB per job), so use `MAX_PARALLEL` ≤ 8 for panoramas

6 Stage 4 – making and viewing an animation

Something like (replace `renders_particles_mist` with the folder for your mode, and `frame` with the prefix of your camera, e.g. `fish` or `vr360_3d`):

```
ffmpeg -framerate 30 -i renders_particles_mist/frame_%04d.png \
       -c:v libx264 -pix_fmt yuv420p -crf 18 giant_impact.mp4
```

For a sequence that does not start at frame 0–4 (e.g. a test of frames 300–319), add `-start_number 300` before `-i`; a short test is easier to judge at `-framerate 10`. Then

```
mplayer -loop 0 giant_impact.mp4
```

For a headset, a stereo `VR360` movie is 4096×4096 (top-bottom); `-c:v libx265 -tag:v hvc1` instead of `libx264` gives a smaller file that headsets decode more readily. Most headset video players recognise the format from the file name, e.g. `giant_impact_360_TB.mp4`; otherwise choose "360°, top-bottom" in the player.

7 Keeping files up to date

You never need to delete files or set flags to make the pipeline redo work. Every file written by stages 1 and 2 carries a *fingerprint*: a short hash of the settings it was made with, stored inside the file itself (a comment line in a `.ply` header, an extra entry in a `.npz`, file metadata in a `.vdb`), so it costs no extra files. Files are written under a temporary name and renamed when complete, so a job that is killed never leaves a half-written file that looks finished. On each run a file is remade only if it is missing or its fingerprint differs from that of the current settings; otherwise the frame is reported as `[up to date]`. The body and rest files are judged separately, so changing `REST_FADE_RHO`, say, remakes only the rest particles and leaves the expensive meshes alone. A `.vdb` file's fingerprint includes that of its volume file, so stage 2 rebuilds exactly the `.vdb` files whose volume was remade or whose `VDB_RES` or `VDB_SMOOTH` changed. This is why the `.vdb` files are made in stage 2: Blender's Python has `openvdb`, and a `.vdb` file survives any change to the body or particle look.

Files	Fingerprint includes
all stage-1 files	FORMAT_VERSION, BOX_SIZE, BLENDER_UNIT_M, SURFACE_RHO, MIN_NEIGHBOURS, IRON_ID
meshes	+ SPLASH_*
particles	+ RHO_DEBRIS, RHO_BODY, TARGET_MAX_ID; and REST_FADE_RHO for rest particles
volume files	+ RESOLUTION
.vdb files	volume file's fingerprint, VDB_RES, VDB_SMOOTH

One caution: only the settings listed are tracked, so if you change the code itself (rather than a setting at the top), add one to FORMAT_VERSION to have everything remade. And a file with no fingerprint (made by an older version of the scripts) counts as out of date and is remade.

8 Checklist before a full run

Stage 1: SIM_NAME, BASE_DIR; RENDER_MODE; BOX_SIZE; densities (SURFACE_RHO, REST_FADE_RHO).

stage1_slurm.sh: --array, --cpus-per-task and --mem for the mode (table above); -t; the frame range.

Stage 2: SIM_NAME, BASE_DIR and RENDER_MODE as in stage 1; MAKE_TEMPLATE = True; CAMERA_MODE; the look parameters for your mode.

Stage 3: BODY_ON/REST_ON; SAMPLES.

render_chunks.sh: SIM_NAME, MODE and CAMERA (the settings most easily forgotten when switching); START_FRAME, END_FRAME, CHUNK; MAX_PARALLEL for the GPU memory free; MAX_PARALLEL×NTHREAD $\leq$ 128.

Changed	Re-run
Density thresholds, box, mesh or mist-grid settings	stages 1 → 2 → 3 (only affected files are remade)
Colours, glow, particle size, alpha, mist look, camera, VDB_RES	stages 2 → 3
SAMPLES, components on/off at render time, frame range	stage 3 only

9 The most useful Blender controls

This section is for taking a saved .blend and adjusting it. Open the .blend saved by a stage-2 *test* run (MAKE_TEMPLATE = False), which still contains the test frames; the template contains no per-frame geometry, so there is nothing to see in it until stage 3 imports a frame. Changes you make by hand are not picked up by the scripts, so once you have found settings you like, copy them into the stage-2 parameters. Click-paths are for Blender 5.1; menu labels move slightly between versions, so verify against your build. Where a name is version-sensitive it is flagged.

9.1 Rendered view, and EEVEE vs Cycles

The four shading buttons at the top-right of the 3-D viewport are Wireframe, Solid, Material Preview, and **Rendered** (rightmost). Click Rendered to see lights, emission and bloom; press Z for the same as a pie menu.

Choose the engine in **Properties** → **Render Properties** (the camera-back icon in the column of icons tabs running down the right-hand-side of your screen) → *Render Engine*:

- **EEVEE** – fast renderer, near real-time; stage 2 uses it for test renders except with the fisheye camera. Good for previewing and for standard flat renders.
- **Cycles** – path-traced; slower but physically accurate. Stage 3 always renders with Cycles on the GPU. **Required for the planetarium fisheye/dome**, because the panoramic/fisheye lens is a Cycles feature. For a fulldome master: switch to Cycles, select the camera (in the top-right object list to highlight it), **Camera Data Properties (the green camera icon on the right column) → Lens → Type: Panoramic → Panorama Type: Fisheye Equidistant**, and set a square resolution (e.g. 4096×4096, as `CAMERA_MODE = "FISHEYE"` does).

9.2 Rendering: single frame, animation, output, resolution, samples

- **Single frame: Render → Render Image** (`F12`). **Animation: Render → Render Animation** (`Ctrl+F12`).
- **Where files go: Output Properties** (printer icon) → the output path field. Single-image renders open in a window and are saved with *Image → Save As*; animations write to the path automatically.
- **Resolution / aspect ratio: Output Properties → Format → Resolution X/Y** and the % scale. Aspect ratio is X:Y (the scripts use 1280×720); use a square ratio for dome output.
- **File format: Output Properties → Output → File Format** – *PNG* for a frame sequence, or *FFmpeg Video* for a direct MP4 (see section 6).
- **Samples (noise vs speed): Render Properties → Sampling**. In Cycles raise *Render* samples to reduce noise; in EEVEE the equivalent is *Samples*. Cycles also has a *Denoise* toggle worth enabling.

9.3 Cameras, including an orbiting camera

The scene *already contains a camera* called `Cam` (the one the script set up). The active camera is the one with the filled triangle marker in the **Outliner** (top-right object list); to make a selected camera active, **View → Cameras → Set Active Object as Camera** (`Ctrl+Numpad 0`). Look through it with `Numpad 0`.

Aim it by hand: navigate the viewport to the angle you want, then **View → Align View → Align Active Camera to View** (`Ctrl+Alt+Numpad 0`). Fine-tune in the `N` panel (*Item → Transform*). The scene is centred on the origin, **(0, 0, 0)**, and lengths are in the simulation's length unit (Earth radii here). Note that the script keyframes the camera's location and rotation, so a hand-aimed camera is overridden on other frames unless you re-key it or delete those keyframes.

Orbit it around the collision. The reliable recipe is to pivot the camera around an empty at the centre, *and* lock it to look at that empty so it can never tilt off on its own:

1. **Add → Empty → Plain Axes.** In the `N` panel set its *Location* to **(0, 0, 0)**. Everything orbits this point, so it must sit at the centre, not on the camera.
2. **Point the camera at the empty.** Select the camera, open **Object Constraint Properties** (the chain-link icon in the right-hand tab column, a few below the orange Object square), **Add Object Constraint → Track To**. Set *Target* = the empty, *Track Axis* = `-Z`, *Up* = `Y`. The

camera now always faces the centre – this is what stops the "looking up, back and down" tumble.

3. **Parent the camera to the empty.** Click the camera, then `Shift`-click the empty (order matters – the *last* clicked becomes the parent), and press `Ctrl+P` → **Object (Keep Transform)**. Spinning the empty now carries the camera with it.

4. **Animate the spin – about the vertical Z axis.** Select the empty. On frame 1, in the `N` panel set *Rotation Z* = 0, right-click that field → *Insert Single Keyframe*. Go to the last frame, set *Rotation Z* = 360, right-click → *Insert Single Keyframe*. Use Z (the blue up-axis): animating X or Y instead is exactly what rolls the camera up and over rather than circling it.

5. **Make the speed constant.** In a **Graph Editor**, select the rotation curve and set **Channel** → **Interpolation Mode** → **Linear** (`T` → Linear).

If it just spins on the spot: the empty is not at the centre – it is sitting on the camera, so the orbit radius is zero; fix its location to (0, 0, 0). Or the camera was not actually parented – re-do step 3 with the empty clicked *last*; or you keyed X/Y rotation instead of Z. The Track-To constraint in step 2 guarantees the camera keeps facing the collision, so once it is orbiting it cannot pitch away.

9.4 Viewing in VR

Blender's VR is the built-in **VR Scene Inspection** add-on (OpenXR), for looking around the scene from inside a headset. Enable and start it as follows:

1. **Edit** → **Preferences** → **Add-ons**, search *VR*, and tick **3D View: VR Scene Inspection**.

2. Open the viewport `N` panel; a new **VR** tab appears. Under it, click **Start VR Session**.

This needs an OpenXR runtime (SteamVR, Meta/Oculus, or Windows Mixed Reality) and a connected headset; it is best supported on Windows, with Linux OpenXR (e.g. Monado) still experimental. VR inspection runs in the *real-time viewport*, so it uses EEVEE, not Cycles – it is a separate thing from the offline Cycles fisheye render used for a fulldome master.

9.5 Lighting

For flat cameras (`PERSP`, `ORTHO`) the scene contains a **key sun** (`KeySun`, energy 4.0) and a dimmer **fill sun** (`FillSun`, energy 1.2), turned with `CAMERA_VIEW` so they light the bodies the same way from any viewing side. A dome or headset view sees bodies in widely different directions, where fixed suns would light each from a different side; for `FISHEYE` and `VR360` the suns are replaced by a lamp at the camera (`CameraLamp`), which lights everything from the viewer's direction. `LAMP_STRENGTH` (3, in sun units) sets how strongly; its power is keyframed with the camera path so the bodies stay equally bright as the camera moves in and out. Select a light in the **Outliner**, then **Object Data Properties** (the lightbulb icon) to change *Strength*/energy and, for suns, *Angle* (softness of shadows). Rotate a sun in the `N` panel (*Item* → *Rotation*) to move the highlight across the body. Add more with **Add** → **Light** → **Area/Point/Sun**. Because the material is also self-emissive, lights mainly shape the lit rock; raising `BODY_EMIT_GAIN` or `EMIT_GAIN` in stage 2 has more effect on the molten glow.

9.6 Changing materials safely, and re-rendering

The materials are `BodyLook` (mesh body), `ParticleLook` (all particles) and `MistLook` (volume); a template contains those its mode uses. Select an object, open the **Shading** workspace (top tab) or **Material Properties** (checkerboard-sphere icon) to see the node graph. Particles get their material from the *Set Material* node inside their Geometry Nodes modifier rather than from the object's material slot.

- **Glow / opacity / size** are Blender-side: adjust the emission chain (the *Math: Multiply* node feeding *Emission Strength*, i.e. `BODY_EMIT_GAIN` / `EMIT_GAIN`), or edit the stage-2 parameters and re-run stage 2 only. No stage-1 rerun needed.
- **Colour** for every material is in the `HeatLook` node group (open it with `Tab` on the group node): the rock and iron colours, the palette ramp and the glow range. A change there applies to body, particles and mist at once. To make it permanent, copy it into the stage-2 parameters (`ROCK_RGB`, `PALETTE`, `GLOW_K`).
- **Re-render** after any change with `F12` (frame) or `Ctrl+F12` (animation). Save incremental `.blend` copies so a tweak is easy to undo.

Version notes. The scripts are written for Blender 5.x: they use the `wm.ply_import` importer and handle the layered actions of 4.4+ when keyframing visibility. This matters only if you script further.

A macOS / Windows equivalents

Everything above is Linux-first. The only differences are the shell and the Blender executable path.

Environment activation: conda is identical on all platforms (`conda activate giantimpact`); for a venv, Windows uses `giantimpact\Scripts\activate`.

Blender executable (replace `blender` in the stage-2 commands):

```
Linux   :  blender
macOS   :  /Applications/Blender.app/Contents/MacOS/Blender
Windows:  "C:\Program Files\Blender Foundation\Blender 5.1\blender.exe"
```

On Windows, match the version number in the path to your install; use the Anaconda Prompt as the shell.